

Saga, Eventual Consistency, and Outbox Pattern

1. The Problem: Distributed Transactions

In a monolith, you might have:

```
Order Service
  ↓
Database
  ↓
Order + Payment + Inventory
```

One database transaction can potentially handle everything:

```
@Transactional
public void placeOrder() {
    createOrder();
    reserveInventory();
    createPayment();
}
```

If something fails:

```
Any failure
  ↓
Rollback
  ↓
All changes undone
```

Now split this into microservices:

```
Order Service    → Order DB
  ↓
Payment Service  → Payment DB
  ↓
Inventory Service → Inventory DB
```

You now have **multiple databases and multiple transaction boundaries**.

A single local:

```
@Transactional
```

does not automatically make all three operations atomic.

This is the problem that **Saga** and related patterns address.

2. What is Eventual Consistency?

In a distributed system, all services do not necessarily become consistent **at exactly the same instant**.

Example:

```
Order Service
Order = CREATED ✓

Payment Service
Payment = PROCESSING

Inventory Service
Reservation = NOT_YET_DONE
```

For a short period, different services see different states.

Then:

```
Payment completed
  ↓
Inventory reserved
  ↓
All services converge
```

This is **eventual consistency**.

Important

It does **not** mean:

“The data can remain inconsistent forever.”

It means:

The system is temporarily inconsistent but is designed to converge to a consistent state.

3. Why Eventual Consistency?

Because distributed systems cannot always provide the same simple atomic transaction semantics as one database.

You have trade-offs involving:

- availability
- latency
- network failures
- independent service ownership
- distributed transactions

Instead of trying to make every service commit at exactly the same moment, we design the workflow so that the system can **recover from partial failure**.

4. What is Saga?

A **Saga** is a sequence of local transactions across multiple services.

Each service:

1. performs its own local transaction
2. triggers the next step
3. if a later step fails, compensating actions undo/reverse earlier business effects where possible

Example:

```
Create Order
  ↓
Reserve Inventory
  ↓
Process Payment
```

If payment fails:

```
Payment FAILED
  ↓
Cancel Inventory Reservation
  ↓
Cancel Order
```

These are **compensating transactions**.

5. Important: Saga Does NOT Mean Database Rollback

This distinction is critical.

A normal transaction:

```
DB Transaction
  ↓
rollback()
```

actually rolls back database changes atomically.

A Saga does:

```
Step 1 succeeds
Step 2 succeeds
Step 3 fails

  ↓

Execute compensating action
```

You're not rewinding history.

You're performing a **new business operation that compensates for the previous operation**.

6. Example

Suppose:

```
Order = CREATED
Inventory = RESERVED
```

```
Payment = FAILED
```

You can't magically restore the database state across all services.

Instead:

```
Payment failed
  ↓
Inventory Service
  ↓
Release inventory
```

Then:

```
Order Service
  ↓
Cancel order
```

Final state:

```
Order = CANCELLED
Inventory = AVAILABLE
Payment = FAILED
```

This is the Saga approach.

7. Two Types of Saga

There are two major approaches:

```
Saga
├─ Choreography
└─ Orchestration
```

8. Choreography

There is no central coordinator.

Services react to events.

Example:

```
Order Service
  ↓
OrderCreated
  ↓
Kafka
  ↓
Payment Service
  ↓
PaymentCompleted
  ↓
Kafka
  ↓
Inventory Service
```

Each service knows what event to react to.

Conceptually:

```
Order
  ↓ event
Payment
  ↓ event
Inventory
```

Advantage

Loose coupling.

Disadvantage

Business workflow becomes distributed across many event handlers.

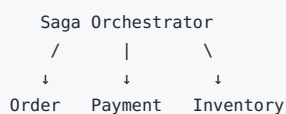
With enough services, understanding:

“What exactly happens to an order?”

can become difficult.

9. Orchestration

A central **Saga Orchestrator** coordinates the workflow.



For example:

```
Orchestrator
  ↓
Create Order
  ↓
Reserve Inventory
  ↓
Process Payment
```

If payment fails:

```
Payment FAILED
  ↓
Orchestrator
  ↓
Release Inventory
  ↓
Cancel Order
```

Advantage

The business workflow is visible in one place.

Disadvantage

The orchestrator becomes an important coordination component and can introduce coupling if

badly designed.

10. Choreography vs Orchestration

Choreography	Orchestration
Event-driven	Central coordinator
No central workflow controller	Orchestrator controls flow
Services react to events	Services execute commands
Can be loosely coupled	Easier to understand workflow
Can become hard to trace	Orchestrator can become complex

11. A Payment Example

Suppose:

```
Order
↓
Payment
↓
Inventory
```

Orchestrated Saga:

```
Saga Orchestrator
↓
Create Order
↓
Reserve Inventory
↓
Process Payment
↓
SUCCESS
```

If payment fails:

```
Payment FAILED
↓
Release Inventory
↓
Cancel Order
```

That is a business-level recovery workflow.

12. What is the Outbox Pattern?

Now suppose you have this problem:

A service needs to:

1. update its database

2. publish an event

Example:

```
Payment Service
├─ DB → payment = SUCCESS
└─ Kafka → PaymentCompleted
```

What if:

```
DB update ✓
Kafka publish ✗
```

Now you have:

```
Database says SUCCESS
Kafka has no event
```

Other services never hear about the payment.

This is called the **dual-write problem**.

13. Dual-Write Problem

Naive implementation:

```
@Transactional
public void completePayment() {

    paymentRepository.save(payment);

    kafkaTemplate.send(
        "payment-completed",
        event
    );
}
```

These are two different systems:

```
Database
Kafka
```

There is no normal single local transaction that automatically makes both operations atomic.

You can get:

```
DB ✓
Kafka ✗
```

or, depending on the failure:

```
Kafka ✓
DB ✗
```

That is dangerous.

14. Outbox Pattern Solution

Instead of directly publishing the event, save the event in the **same database transaction** as the business update.

Payment DB

```
| payment |
| id=100  |
| status=SUCCESS |
|
| outbox_event |
| id=EVT100    |
| type=PaymentCompleted |
| payload={...} |
```

Both writes happen in one local DB transaction:

```
BEGIN
↓
Update payment
↓
Insert outbox event
↓
COMMIT
```

Now you have atomicity **within the database**.

15. Then Publish the Outbox Event

A separate process reads the outbox table:

```
Outbox Table
↓
Outbox Publisher
↓
Kafka
↓
Consumer
```

Flow:

```
Payment Service
↓
DB transaction
| Payment |
| Outbox Event |
↓
Publisher
↓
Kafka
↓
Other Services
```

16. Why This Solves the Dual-Write Problem

Suppose database transaction succeeds:

```
Payment = SUCCESS
Outbox Event = STORED
```

But Kafka is temporarily unavailable.

That's okay.

The event remains in the outbox:

```
Outbox
↓
Kafka unavailable
↓
retry later
```

Eventually:

```
Outbox → Kafka ✓
```

You don't lose the event just because Kafka was temporarily unavailable.

17. But Doesn't Outbox Still Have Duplicates?

Yes.

Suppose:

```
Publisher sends event to Kafka ✓
Publisher crashes before marking outbox record as sent
```

On restart:

```
Outbox publisher
↓
sees event as unsent
↓
publishes again
```

Now:

```
EVT100
EVT100
```

may be delivered.

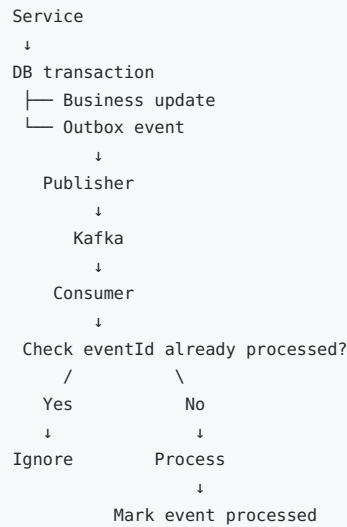
Therefore the consumer should generally be **idempotent**.

This is an important point:

Outbox solves reliable event publication, but it does not magically guarantee exactly-once business processing end-to-end.

18. Outbox + Idempotent Consumer

A robust flow:



This handles duplicate delivery.

19. Outbox Table Example

A simple schema might look like:

```
OUTBOX_EVENT

id
aggregate_id
event_type
payload
created_at
published_at
```

Example:

```
id          = EVT1001
aggregate_id = PAY123
event_type  = PaymentCompleted
payload     = {...}
published_at = NULL
```

After successful publication:

```
published_at = 2026-09-11T10:30:00
```

The exact implementation varies.

20. Polling vs CDC

There are different ways to move outbox events to the broker.

Polling

Publisher periodically queries:

```
SELECT *  
FROM outbox_event  
WHERE published_at IS NULL;
```

Then sends them to Kafka.

CDC — Change Data Capture

A CDC tool reads database changes and publishes them.

A common ecosystem example is:

Debezium

Conceptually:

```
Database  
↓  
CDC  
↓  
Kafka
```

This avoids having application code continuously poll the database.

You don't need to implement CDC for a 3-year interview, but know why it can be used.

21. Saga + Outbox Together

These patterns often work together.

Example:

```
Order Service  
↓  
DB transaction  
├─ Order = CREATED  
└─ Outbox: OrderCreated  
    ↓  
    Kafka  
    ↓  
    Payment Service  
    ↓  
    local transaction  
    ├─ Payment = SUCCESS  
    └─ Outbox: PaymentCompleted  
        ↓  
        Kafka  
        ↓  
        Inventory Service
```

If payment fails:

```
PaymentFailed  
↓  
Saga workflow  
↓  
Compensating action  
↓
```

Cancel Order

So the roles are different:

Saga
→ Coordinates distributed business transaction

Eventual Consistency
→ Accepts temporary inconsistency while services converge

Outbox
→ Reliably publishes events associated with local DB changes

22. Very Important Distinction

These three are **not interchangeable**.

Eventual Consistency

A **consistency model**.

Saga

A **distributed transaction/workflow pattern**.

Outbox

A **reliable messaging/persistence pattern**.

Remember:

```
Eventual Consistency
  ↑
overall model

Saga
  ↑
distributed workflow

Outbox
  ↑
reliable DB → event publication
```

23. Why Not Use Distributed Transactions / 2PC?

The interviewer may ask:

“Why not use two-phase commit?”

Conceptually, 2PC:

```
Coordinator
 /      \
DB1      DB2
```

tries to coordinate commits across systems.

But in modern microservice architectures, many teams avoid it because it can introduce:

- blocking
- coordination overhead
- availability concerns
- operational complexity
- tight coupling between systems

Instead, systems often use:

```
Saga
+
Eventual consistency
+
Outbox
```

when the business domain permits it.

Don't claim Saga is always superior. The requirements determine the choice.

24. Saga vs @Transactional

This is a very important interview question.

Local transaction

```
@Transactional
```

works within the transaction manager/resource scope configured for that operation.

It can provide atomicity for a local database transaction.

Saga

Coordinates multiple local transactions:

```
Service A transaction
    ↓
Service B transaction
    ↓
Service C transaction
```

with compensation where needed.

So:

```
@Transactional and Saga solve different scopes of the consistency problem.
```

25. Payment Example

Suppose you have:

```
Payment Service
```

```
↓
Billing Service
↓
History Service
```

Imagine:

```
Payment SUCCESS
Billing update SUCCESS
History update FAIL
```

A distributed transaction doesn't simply roll everything back with one local annotation.

You need a business decision:

```
Retry history
    OR
Compensate
    OR
Mark transaction for reconciliation
```

A Saga could coordinate compensating actions where appropriate.

An Outbox can make sure important events aren't lost.

Eventual consistency acknowledges that these service states may temporarily differ.

26. How This Connects to Payment Systems

Payment systems are particularly sensitive because compensation isn't always equivalent to rollback.

Example:

```
External Payment Provider
↓
₹5,000 successfully charged
```

Then an internal service fails.

You cannot simply:

```
database.rollback()
```

and assume the external payment disappears.

You may need:

```
refund
reversal
status inquiry
reconciliation
```

That's why **Saga design in financial/payment systems requires careful domain-specific compensation.**

This is an excellent interview point.

27. Common Interview Scenario

Question:

“Order creation succeeded but payment failed. How do you maintain consistency?”

A good answer:

```
Order = CREATED
↓
Payment = FAILED
↓
Saga detects failure
↓
Compensating action
↓
Order = CANCELLED
```

But if payment had actually succeeded and only the response was lost:

```
Payment provider = SUCCESS
BPS = TIMEOUT
```

you might need:

```
status inquiry
+
reconciliation
```

rather than simply compensating immediately.

This distinction shows strong distributed-system thinking.

28. Common Interview Scenario — Outbox

Question:

“How do you guarantee that when an order is saved, the `OrderCreated` event isn’t lost?”

A strong answer:

“I’d use the transactional outbox pattern. In the same local database transaction that persists the order, I’d insert an outbox record containing the event. A separate publisher reads unpublished outbox records and sends them to the message broker. If the broker is temporarily unavailable, the event remains in the outbox and can be retried. Since publication can still result in duplicate delivery, consumers should process events idempotently.”

That’s a **very good 3-year-level answer**.

29. Common Interview Scenario — Saga

Question:

“Payment succeeded but inventory reservation failed. What do you do?”

Answer:

```
Payment = SUCCESS
Inventory = FAILED
  ↓
Saga detects failure
  ↓
Compensation
  ↓
Refund/reverse payment
  ↓
Cancel order
```

But add:

“The exact compensating action depends on the business semantics. For a payment, compensation may be a refund or reversal rather than a database rollback.”

Excellent distinction.

30. Interview Questions You Should Prepare

Eventual Consistency

1. What is eventual consistency?
2. Why is it needed in microservices?
3. Give a real example.
4. Immediate consistency vs eventual consistency?
5. What happens if a consumer is temporarily unavailable?
6. How do you handle temporarily inconsistent data?

Saga

7. What is the Saga pattern?
8. Why do we need Saga?
9. Saga vs database transaction?
10. What is a compensating transaction?
11. Choreography vs orchestration?
12. When would you use each?
13. What happens if compensation itself fails?
14. Can every operation be compensated?
15. How would you use Saga for payment processing?

Outbox

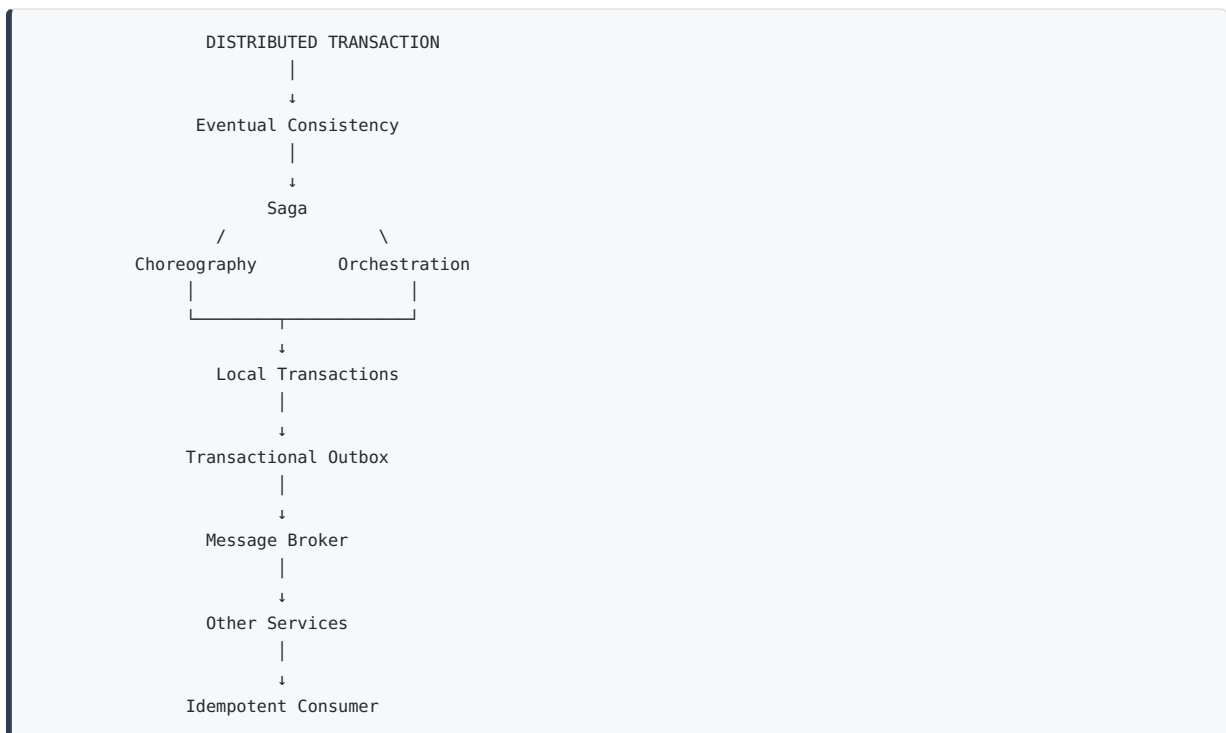
16. What problem does Outbox solve?
17. What is the dual-write problem?
18. How does transactional outbox work?

19. Why save the outbox event in the same transaction?
20. How does the publisher work?
21. What happens if Kafka is unavailable?
22. Can Outbox produce duplicates?
23. How do you make consumers idempotent?
24. Polling vs CDC?
25. What is Debezium?

Architecture

26. Saga + Outbox relationship?
27. Outbox vs 2PC?
28. Saga vs `@Transactional` ?
29. How do you handle distributed transaction failures?
30. How do you monitor/recover failed Saga steps?

31. One Diagram to Remember



32. Interview-Ready Summary

“In a microservices architecture, a business transaction can span multiple services and databases, so a single local `@Transactional` transaction cannot provide atomicity across the entire workflow. Eventual consistency allows individual services to commit independently and converge to a consistent business state. The Saga pattern manages such a distributed workflow as a sequence of local transactions, with compensating transactions used when later steps fail. Saga can be implemented through choreography, where

services react to events, or orchestration, where a central coordinator manages the workflow. The transactional Outbox pattern solves the dual-write problem between a service's database and its message broker by storing the business change and its outgoing event in the same local database transaction, then publishing the event asynchronously. Because publishing can still result in duplicate delivery, consumers should be idempotent."

Your 3-year priority

Learn these in this order:

1. Distributed transaction problem
2. Eventual consistency
3. Saga
4. Compensation
5. Choreography vs Orchestration
6. Dual-write problem
7. Outbox pattern
8. Idempotent consumer
9. Retry + DLQ
10. Outbox + Kafka
11. 2PC vs Saga
12. Payment-specific compensation

BPS questions to investigate in your codebase

1. Where does one BPS business operation cross multiple systems?
2. Which operations are local DB transactions versus remote operations?
3. What happens when a downstream operation succeeds but the next BPS step fails?
4. Does BPS use any compensating operation, reversal, retry or reconciliation mechanism?
5. Is any event/message published after a database update?
6. If yes, how does the code guarantee the event isn't lost?
7. Is there an outbox table/publisher anywhere in the architecture?
8. Which BPS operations eventually become consistent rather than immediately consistent?
9. For an online payment, what happens if the external provider succeeds but BPS does not receive the response?
10. Which parts of that scenario are handled by transaction management, and which require reconciliation?